

Relatório 11 - Vídeo e Leitura: Infraestrutura com AWS Part 2 (IV)

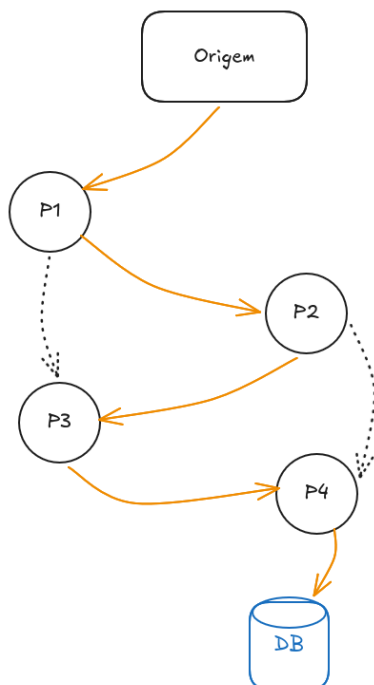
José Carlos Seben de Souza Leite

Descrição da atividade

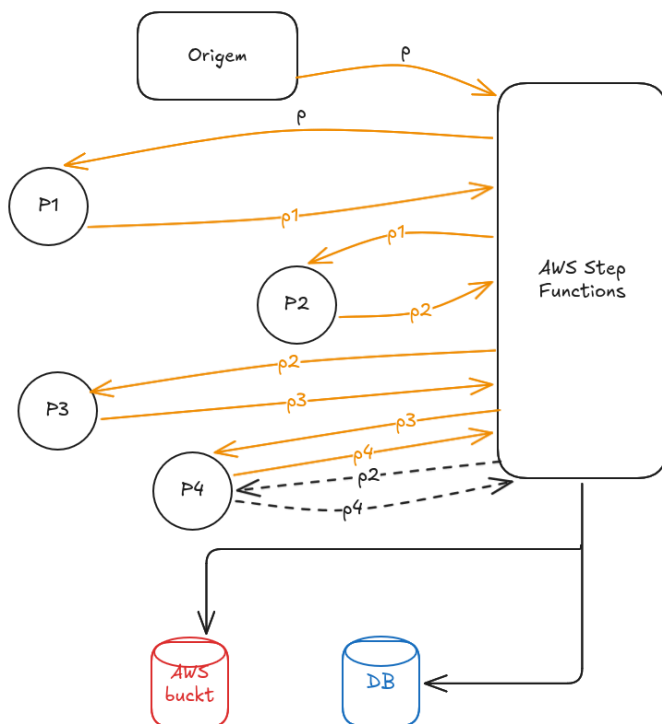
Começamos com o conteúdo sobre AWS step functions no vídeo “**O que é e para que serve o AWS Step Functions ?**” que basicamente é um orquestrador de mensagem responsável por gerenciar a comunicação de microsserviços. Como no exemplo situado no vídeo onde teríamos uma situação onde teríamos 4 microsserviços que se comunicaram um com o outro sendo em uma comunicação quase linear de P1, para P2 e assim por diante, mas também dependendo da situação empregada terias exemplos de P1 se comunicar diretamente com P4 sem passar por P2, e P3, em situações como esta que o **AWS Step Functions** brilha por ele ser algo como um centro de comunicação onde os microsserviços não vão se comunicar diretamente um com o outro, mas sim com ele então uma requisição iria para P1 e de P1 para o Step Functions, se necessário passar por outros microsserviços aconteceria a mesma comunicação agora pegando a resposta de P1 a tratando se necessário e enviando para P2 e recebendo a resposta de P2 assim todas as comunicações passando pelo mesmo.

Outra coisa interessante sobre o mesmo é sua capacidade de realizar tarefas simultâneas, como no exemplo que vai estar a baixo ao mesmo tempo que vamos estar salvando algo em um banco de dados vai estar sendo salvo os mesmos dados em um S3 bucket da AWS, permitindo otimizar o trabalho, reduzir as quantidades de código e até mesmo diminuir os custos referentes a controle da AWS na aplicação se bem implementado.

Nesse exemplo a baixo é de um sistema padrão sem o uso do AWS Step Functions, onde os microsserviços estão representados pelos círculos laranjas de P1 a P4 e as comunicações diretas representadas pelas flechas diretas, e comunicações que pulam etapas por flechas com linhas pontilhadas.



E a baixo uma representação da comunicação de um sistema com o AWS Step Functions.



Agora entrando no conteúdo sobre AWS RDS com base no conteúdo do vídeo “**O Que é AWS RDS - Vantagens e Banco de Dados**” basicamente podemos ver como um modo de aplicarmos e gerenciarmos um banco de dados, mas com esse serviço tiramos a complexidade de gerenciamento de máquina e algumas outras coisas de nossa responsabilidade e deixamos por gerenciamento da própria AWS, pois o **RDS** é um serviço que pagamos para receber essa “ajuda” e alguns outros benefícios que podem nos ajudar como a segurança da AWS, como facilmente podermos manter uma ou mais cópias de nosso banco de dados em diferentes locais para no caso de uma máquina dar problema termos um safe spot de confiança que dos dados da empresa não serão perdidos.

Sobre o conteúdo de **DynamoDB** do vídeo “**DynamoDB: o que é e como utilizar? Domine de vez o assunto**” que é basicamente uma explicação teórica sobre ele onde é apresentado que o DynamoDB é um banco de dados não relacional, que funciona através do modelo de chave-valor, nele trabalhamos com três coisas básicas, as tabelas, os itens e os atributos. A grande vantagem do DynamoDB é que ele é um serviço **Serverless**, ou seja, a gente não precisa se preocupar em gerenciar servidores, memória ou o tamanho do disco, porque a própria AWS cuida disso e faz o banco crescer e escalar de forma automática conforme a nossa necessidade. Outro ponto importante abordado no vídeo é a alto desempenho, já que ele consegue fazer buscas em uma velocidade de milissegundos por não ter a complexidade de um banco relacional.

Entrando no conteúdo sobre **AWS SQS** com base no vídeo “**Fila de mensagens como serviço | AWS SQS Overview**” onde diferenciamos Broadcast da fila de mensagens, onde basicamente o broadcast pega uma mensagem e a réplica para todas as máquinas conectadas mandando a mesma mensagem para cada uma das máquinas igualmente, enquanto a fila de mensagem faz praticamente o inverso criando a fila e mandando sequencialmente cada mensagem para uma máquina. Sendo versátil para sistemas que aceitam pagamento usando como uma camada de segurança assim garantindo que se mesmo que a máquina que esteja processando o pagamento passe por algum problema não perdemos o mesmo e os seguintes, além de podermos configurar o sistema de filas first in first out permitindo que sempre o

primeiro a entrar vai ser o primeiro a seguir mantendo a ordem original de processamento bastante usado para computação de tokens e coisas desse tipo.

Além disso, destaca que o **SQS** permite o processamento assíncrono, ou seja, o usuário não precisa ficar esperando a tarefa terminar para receber uma resposta da aplicação, o que melhora a experiência de uso, outro ponto importante é a alta disponibilidade e redundância, pois o **SQS** replica as mensagens em vários locais para garantir que elas não sejam perdidas caso um servidor falhe. Também foi mencionado que o sistema ajuda a economizar recursos, pois evita que a gente precise escalar toda a infraestrutura da API só para aguentar picos de acessos, como numa Black Friday, já que a fila segura o volume e vai entregando as mensagens para os "workers" processarem no tempo deles sem derrubar o sistema.

Entrando no conteúdo sobre **AWS CloudWatch Logs** com base no vídeo "**Basics of AWS CloudWatch Logs**", onde aprendemos sobre o gerenciamento e monitoramento de logs dentro da AWS, basicamente podemos ver esse serviço como um centralizador de histórico de tudo o que acontece na nossa aplicação e nos serviços conectados. A vantagem é que, em vez de termos que acessar máquina por máquina para caçar erros ou entender o comportamento do sistema, o CloudWatch Logs puxa esses registros automaticamente de servidores como EC2, funções Lambda ou bancos de dados, guardando tudo em um único lugar fácil de pesquisar.

Além disso, o sistema se mostra muito versátil para solucionar problemas rapidamente, funcionando como uma camada de diagnóstico em tempo real. Se o nosso sistema de pagamentos ou de tokens falhar, por exemplo, conseguimos configurar métricas e alarmes baseados nos padrões desses logs para nos avisar na hora se algo der errado. Para fechar, também podemos definir regras de retenção para esses dados, escolhendo por quanto tempo queremos guardar esse histórico antes de apagá-lo, o que ajuda a economizar com armazenamento na nuvem sem perder o controle dos eventos da empresa.

Entrando agora no último conteúdo relevante do card sobre **Amazon CloudFront** utilizando como fonte "**O que é o Amazon CloudFront?**" basicamente podemos ver como "pontos de distribuição" no caso quando utilizamos ele para compartilhar algo na nuvem ele será responsável por disponibilizar o conteúdo no ponto que permita o usuário que fazer a requisição obter este recurso no menor tempo possível, ou seja, o Amazon CloudFront é responsável por "mandar" esses dados para o ponto em que a resposta para a requisição tenha a menor latência possível assim dependendo de onde esta sendo feita a requisição vai mudar o ponto de "Armazenamento".

Dificuldades

Conclusões

Assim como o card anterior sobre conteúdo da AWS podemos concluir que com eles conseguimos construir uma base teórica sobre os conteúdos, e me gerou um interesse pessoal sobre o tema chamando atenção por a quantia de diferentes coisas que podemos fazer e automatizar utilizando esses serviços.

Referencias

videos:

 O que é e para que serve o AWS Step Functions ?

 O Que é AWS RDS - Vantagens e Banco de Dados

▶ DynamoDB: o que é e como utilizar? Domine de vez o assunto

▶ Fila de mensagens como serviço | AWS SQS Overview

▶ Basics of AWS CloudWatch Logs

▶ AWS Amplify: Deploy de Uma Aplicação Next.js

Doc:

[What is Amazon CloudFront? - Amazon CloudFront](#)